

Repositorio Genérico (BaseRepository)

Objetivos

- Implementar un repositorio genérico reutilizable
- Usar Python Generics para tipado fuerte
- Evitar duplicación de código CRUD
- Crear la base para repositorios específicos

El Problema: Código Duplicado

Sin un repositorio genérico, cada entidad tiene código repetido:

```
# ❌ Duplicación en cada repositorio
class AuthorRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, id: int) -> Author | None:
        return self.db.get(Author, id)

    def get_all(self, skip: int = 0, limit: int = 100) -> list[Author]:
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def create(self, entity: Author) -> Author:
        self.db.add(entity)
        self.db.flush()
        return entity

    def delete(self, id: int) -> bool:
        entity = self.get_by_id(id)
        if entity:
            self.db.delete(entity)
            return True
        return False

class PostRepository:
    def __init__(self, db: Session):
        self.db = db

    # ❌ Mismo código, diferente modelo
    def get_by_id(self, id: int) -> Post | None:
        return self.db.get(Post, id)

    def get_all(self, skip: int = 0, limit: int = 100) -> list[Post]:
        stmt = select(Post).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

# ... mismo patrón repetido
```

Solución: BaseRepository con Generics

Python Generics

Los **Generics** permiten crear clases que trabajan con cualquier tipo:

```
from typing import TypeVar, Generic

# TypeVar define un tipo variable
T = TypeVar("T")

# Generic[T] indica que la clase usa ese tipo
class Container(Generic[T]):
    def __init__(self, item: T):
        self.item = item

    def get(self) -> T:
        return self.item

# Uso con tipos específicos
int_container = Container[int](42)
str_container = Container[str]("hello")
```

Implementación de BaseRepository

```
# repositories/base.py
from typing import TypeVar, Generic
from sqlalchemy import select, func
from sqlalchemy.orm import Session

from database import Base

# T debe ser un modelo SQLAlchemy (subclase de Base)
T = TypeVar("T", bound=Base)

class BaseRepository(Generic[T]):
    """
    Repositorio genérico con operaciones CRUD básicas.

    Uso:
        class AuthorRepository(BaseRepository[Author]):
            pass
    """

    def __init__(self, db: Session, model: type[T]):
        """
        Args:
            db: Sesión de SQLAlchemy
            model: Clase del modelo (Author, Post, etc.)
        """
        self.db = db
        self.model = model

    def get_by_id(self, id: int) -> T | None:
        """Obtiene entidad por ID"""
        return self.db.get(self.model, id)
```

```

def get_all(self, skip: int = 0, limit: int = 100) -> list[T]:
    """Lista entidades con paginación"""
    stmt = select(self.model).offset(skip).limit(limit)
    return list(self.db.execute(stmt).scalars().all())

def count(self) -> int:
    """Cuenta total de entidades"""
    stmt = select(func.count()).select_from(self.model)
    return self.db.execute(stmt).scalar() or 0

def add(self, entity: T) -> T:
    """Agrega entidad a la sesión"""
    self.db.add(entity)
    self.db.flush() # Obtiene ID sin commit
    self.db.refresh(entity)
    return entity

def add_many(self, entities: list[T]) -> list[T]:
    """Agrega múltiples entidades"""
    self.db.add_all(entities)
    self.db.flush()
    for entity in entities:
        self.db.refresh(entity)
    return entities

def update(self, entity: T) -> T:
    """Actualiza entidad (ya debe estar en sesión)"""
    self.db.flush()
    self.db.refresh(entity)
    return entity

def delete(self, entity: T) -> None:
    """Elimina entidad"""
    self.db.delete(entity)
    self.db.flush()

def delete_by_id(self, id: int) -> bool:
    """Elimina por ID, retorna True si existía"""
    entity = self.get_by_id(id)
    if entity:
        self.delete(entity)
        return True
    return False

def exists(self, id: int) -> bool:
    """Verifica si existe entidad con ID"""
    return self.get_by_id(id) is not None

```

Creando Repositorios Específicos

Con `BaseRepository`, crear repositorios específicos es simple:

```

# repositories/author_repository.py
from models import Author

```

```

from repositories.base import BaseRepository

class AuthorRepository(BaseRepository[Author]):
    """Repositorio para Author - hereda todo de BaseRepository"""

    def __init__(self, db: Session):
        super().__init__(db, Author)

```

¡Eso es todo! `AuthorRepository` ya tiene todos los métodos CRUD.

Uso Básico

```

# En un endpoint o service
def get_authors(db: Session = Depends(get_db)):
    repo = AuthorRepository(db)

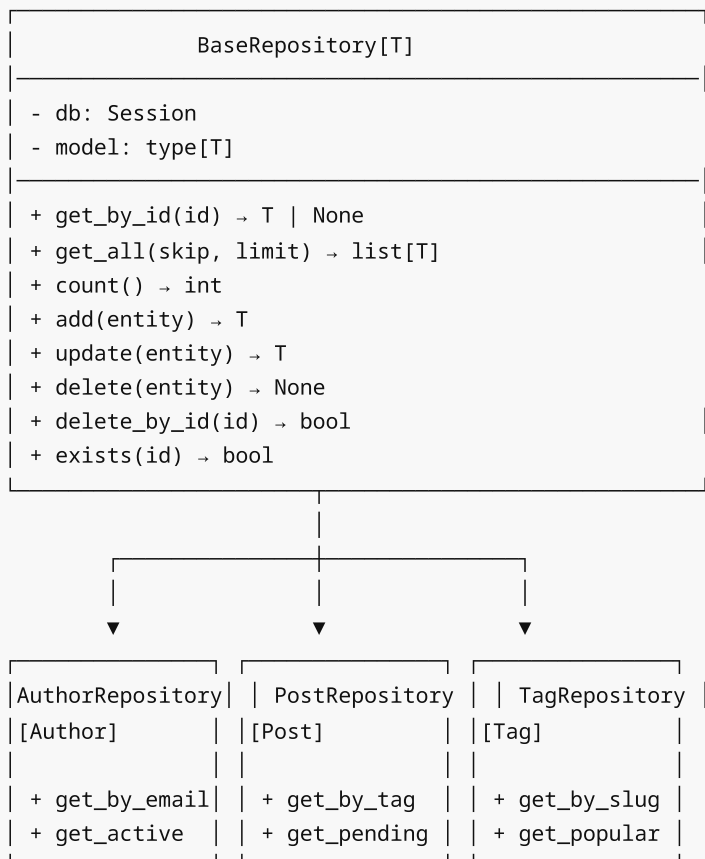
    # Todos estos métodos vienen de BaseRepository
    authors = repo.get_all(skip=0, limit=10)
    author = repo.get_by_id(1)
    total = repo.count()
    exists = repo.exists(1)

    return authors

```



Diagrama de Herencia



Métodos Útiles Adicionales

Puedes extender `BaseRepository` con más métodos genéricos:

```
class BaseRepository(Generic[T]):
    # ... métodos anteriores ...

    def get_or_create(
        self,
        defaults: dict,
        **filters
    ) -> tuple[T, bool]:
        """
        Obtiene entidad existente o crea una nueva.

        Returns:
            tuple: (entidad, created: bool)
        """
        stmt = select(self.model).filter_by(**filters)
        entity = self.db.execute(stmt).scalar_one_or_none()

        if entity:
            return entity, False

        # Crear nueva
        entity = self.model(**filters, **defaults)
        self.add(entity)
        return entity, True

    def get_by_ids(self, ids: list[int]) -> list[T]:
        """Obtiene múltiples entidades por sus IDs"""
        if not ids:
            return []
        stmt = select(self.model).where(self.model.id.in_(ids))
        return list(self.db.execute(stmt).scalars().all())

    def filter_by(self, **kwargs) -> list[T]:
        """Filtra por atributos exactos"""
        stmt = select(self.model).filter_by(**kwargs)
        return list(self.db.execute(stmt).scalars().all())

    def first(self, **kwargs) -> T | None:
        """Obtiene primera entidad que coincida"""
        stmt = select(self.model).filter_by(**kwargs).limit(1)
        return self.db.execute(stmt).scalar_one_or_none()
```

Consideraciones Importantes

1. `flush()` vs `commit()`

```
# flush() - Sincroniza con DB pero NO confirma transacción
self.db.flush() # Obtiene IDs, ejecuta queries, pero puede hacer rollback
```

```
# commit() - Confirma la transacción (permanente)
self.db.commit() # Ya no se puede deshacer
```

En repositorios usamos `flush()` porque el commit lo maneja la capa superior (Unit of Work o el endpoint).

2. Tipado Correcto

```
# ✅ Correcto - tipo específico en herencia
class AuthorRepository(BaseRepository[Author]):
    pass

# ❌ Incorrecto - sin tipo
class AuthorRepository(BaseRepository):
    pass
```

3. Evitar Lógica de Negocio

```
# ❌ MAL - lógica de negocio en repository
class UserRepository(BaseRepository[User]):
    def create_user(self, data: UserCreate) -> User:
        # ❌ Validación de negocio NO va aquí
        if len(data.password) < 8:
            raise ValueError("Password too short")

        # ❌ Transformación de negocio NO va aquí
        user = User(
            email=data.email,
            password=hash_password(data.password) # ← NO aquí
        )
        return self.add(user)

# ✅ BIEN - repository solo persiste
class UserRepository(BaseRepository[User]):
    # Solo métodos de acceso a datos
    def get_by_email(self, email: str) -> User | None:
        return self.first(email=email)
```

✅ Checklist

- ☐ Entiendo cómo funcionan los Generics en Python
- ☐ Sé la diferencia entre `flush()` y `commit()`
- ☐ Puedo crear un repositorio específico heredando de `BaseRepository`
- ☐ Entiendo que la lógica de negocio NO va en repositorios

Siguiente

Aprenderemos a agregar **métodos específicos** a cada repositorio para queries particulares.

→ [03-repositorios-especificos.md](#)